Capstone Scope Document

Author: Derick Haron · **Service:** kk-payments · **Track:** A (Infrastructure-first) · **Date:** 10 November 2026

Problem Statement

Every kk-payments release since Week 8 has been run by hand from a laptop: build the image, compute the semver-SHA tag, push it to the registry, edit the manifest, apply it. There is only one environment, the `default` namespace, so every change goes straight to production with no isolated place to test it against production-like configuration. Configuration is hardcoded in the Deployment manifest, logs are unstructured single lines, and Pods have no readiness probe, so "Ready" means "the process started", not "it can serve payments". Week 9 showed that a bad image makes a rollout **stall, not heal**: someone has to notice it and run the rollback by hand. For a payments service, a release path that depends on a human noticing at 02:00 is the operational gap this capstone closes.

Track

Track A (Infrastructure-first). Production remains the `default` namespace and staging is `kijani-staging`.

What I Will Build

- **Staging environment as code:** Terraform creates the `kijani-staging` namespace with a ResourceQuota, and one Ansible playbook configures **both** environments: the `kk-payments-config` ConfigMap (`APP_ENV`, `DB_HOST=postgres-staging` or `postgres-prod`) and the registry pull secret.
- **One Deployment manifest for both environments:** readiness and liveness probes on `/health`, configuration loaded from the ConfigMap, and the app writing structured JSON logs with a correlation ID on every request.
- **Jenkins delivery pipeline:** on merge to `main`, a dedicated `k8s-deployer` agent builds and pushes `<version>-<sha>`, deploys to staging, runs a smoke test, and only then offers the production approval gate.
- **Automated rollback in staging:** if the rollout does not complete within 3 minutes or the smoke test fails, the pipeline rolls staging back, fails the build, and never reaches production.
- **Log-based error-rate monitor (Option B):** reads kk-payments JSON logs, computes the payment error rate per minute, and writes an `ALERT` to a monitoring summary file when it exceeds 5% for 2 consecutive minutes.

What Is Out of Scope

- **Week 10 serverless receipt chain:** I did not complete Week 10, and a half-built chain would be undemonstrable. It is recorded as a named production gap: the integration point would be a receipts-bucket setting in the same ConfigMap.
- **A Prometheus server:** Option B gives the same observability signal from the existing structured logs without running and securing extra infrastructure on a single-node lab cluster.
- **Ingress and TLS:** NodePort is enough to verify each environment. Controlled public entry belongs to a later phase.
- **A real database:** `DB_HOST` is a configuration value that proves environment separation. kk-payments validates payments and does not yet persist them.
- **Multi-node cluster:** Minikube is single-node, so node-failure recovery cannot be demonstrated honestly.

Success Criteria

1. A merge to `main` deploys the new `<version>-<sha>` to staging with no manual step. The smoke test (`/health` reports that version and `environment: staging`, a valid payment returns 202) passes **before** the approval gate appears. After approval, production `/health` reports the same version with `environment: production`.
2. A deliberately broken release (the app exits at startup because a required setting is missing from the ConfigMap) is rolled back in staging automatically within 3 minutes, the build fails, and production keeps serving the previous version.
3. Sending more than 5% failing payment requests for 2 consecutive minutes makes the monitor write `ALERT`; normal traffic writes `OK`.

Architecture Diagram

Submitted alongside this page as `architecture.png` (also in the repository at `docs/architecture.png`).